

Python Practicals 1–4

Practical 1: Sorting Algorithms

This practical implements Selection Sort, Bubble Sort, Merge Sort, Insertion Sort, and Quick Sort.

Each algorithm includes implementation, time complexity analysis (best, worst, and average cases), and execution time measurement.

Practical 2: Linear Search

This practical implements the Linear Search algorithm with interactive user input.

It demonstrates:

- Implementation of linear search that returns the index of the target (or -1 if not found).
- Measurement of execution time using `time.perf_counter()`.
- Time complexity analysis (Best, Average, Worst cases).

Usage:

- Interactive: run the script and follow prompts to provide the list and target value.
- Demo: run the script with a demo flag (if provided in the script).
- Linear search
- Binary search

Practical 3: Min-Heap and Max-Heap Sort

Description:

This project implements Heap Sort in Python using `heapq`.

Min-Heap: Sorts elements in ascending order.

Max-Heap: Sorts elements in descending order.

Features:

- Takes user input for array elements.
- Uses `heapq.heapify()` and `heapq.heappop()`.
- Measures execution time using `time.perf_counter()`.
- Displays time complexity.

Example:

Min-Heap:

Input: 25, 14, 36, 85, 96

Output: [14, 25, 36, 85, 96]

Max-Heap:

Input: 25, 78, 89, 45, 56, 33
Output: [89, 78, 56, 45, 33, 25]

Complexity:

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

Space Complexity: $O(n)$

Requirements:

Python 3

heapq and time modules (built-in)

Conclusion:

The program demonstrates how Min-Heap and Max-Heap can be used to efficiently sort an array in ascending and descending order.

Practical 4: Factorial Using Iterative and Recursive Function

Description:

This Python program calculates the factorial of a non-negative integer using two methods:

1. Iterative method
2. Recursive method

It also compares the execution time and complexity of both methods.

Example:

Input:
Enter a non-negative integer: 5
Output:
Number: 5

Iterative Method:

Factorial: 120
Execution Time: 0.0000012000 seconds
Time Complexity: $O(n)$
Space Complexity: $O(1)$

Recursive Method:

Factorial: 120
Execution Time: 0.0000015000 seconds
Time Complexity: $O(n)$
Space Complexity: $O(n)$

Complexity:

Method Time Space

Iterative $O(n)$ $O(1)$

Recursive $O(n)$ $O(n)$

Requirements:

Python 3.x

Run:

```
python factorial.py
```

Conclusion:

Both methods produce the same factorial result. The iterative method uses less memory, while the recursive method demonstrates the use of recursion.